

What makes a good prompt?

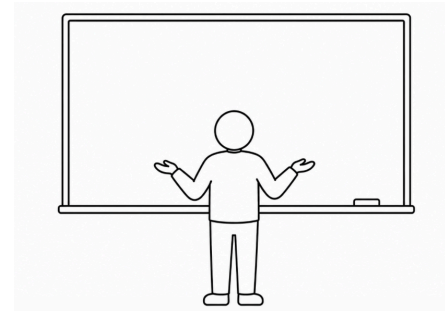
Module 2, Chapter 1

Kirsten Hunter

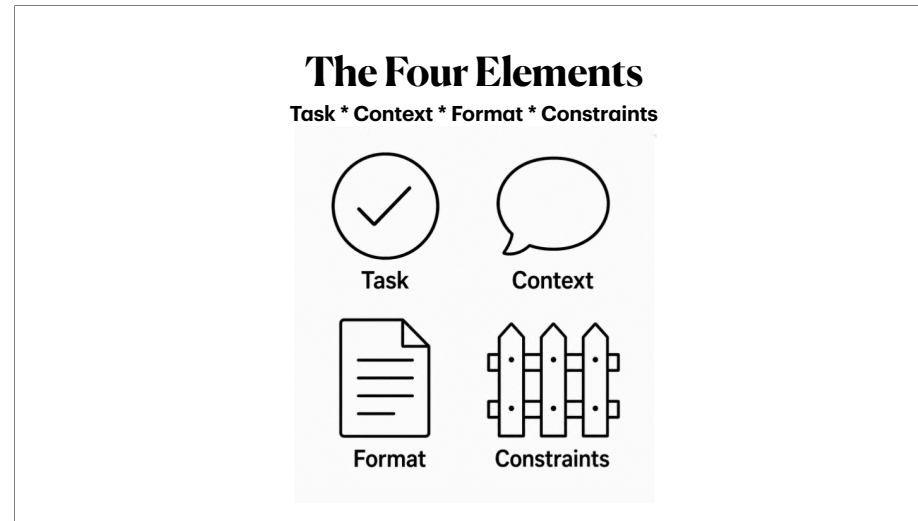
The difference between a useful LLM response and a frustrating one is almost never the model. It's the prompt. In this chapter we're going to break down exactly what makes a prompt work — and give you a framework you can use every time.

No Mind Reading

**It works with what
you give it**



Here's the most important thing to internalize about LLMs: they have no context you didn't provide. No memory of your previous conversations, no idea what industry you work in, no sense of what "good" looks like for your specific use case. Every prompt starts from zero. The model works with exactly what's in front of it — nothing more.



Every strong prompt has most of these four elements. Task — what you want the model to do. Context — the information it needs to do it well. Format — what the output should look like. And constraints — the guardrails that keep the response on target. You don't always need all four, but when a prompt isn't working, one of these is almost always missing.

Task

Be specific

Weak prompt

Summarize this article

Strong prompt

Summarize this article in three bullet points to a non-technical audience

The task is what you're asking for — but vague tasks get vague responses. "Summarize this" and "summarize this in three bullet points for a non-technical audience" are technically the same request. The outputs will be completely different. The more specific you are about what done looks like, the closer the model gets on the first try.

Context

Role background audience



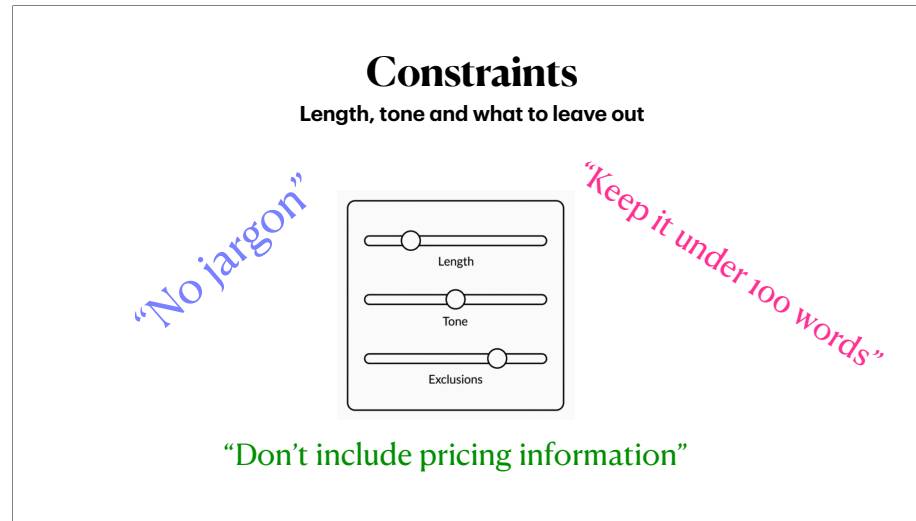
Context is the background information the model needs to give you a relevant answer. Who is this for? What's the situation? What does the reader already know? You might tell it "you're writing for a developer audience that's new to AI" or "this is for an executive summary, not a technical spec." That context changes everything — the vocabulary, the depth, the assumptions it makes.

Format

Avoid the default!

- Numbered steps
- Table
- Code block
- Single paragraph
- Email

If you don't tell the model what format you want, it picks one. Sometimes it guesses right. Often it doesn't. If you needed a table and got three paragraphs, that's not the model failing — that's a missing instruction. Bullet list, numbered steps, table, code block, single paragraph, email — whatever you need, just say so. It takes three extra words and saves a lot of back and forth.



Constraints are the guardrails. They tell the model what not to do as much as what to do. Length limits — "keep it under 100 words" — stop the model from padding. Tone instructions — "plain English, no jargon" — keep it accessible. Exclusions — "don't mention pricing" — keep it scoped. Constraints aren't restrictive, they're clarifying. The tighter the brief, the better the output.

Before and After

Same task, different results

Before

Write a blog post about AI

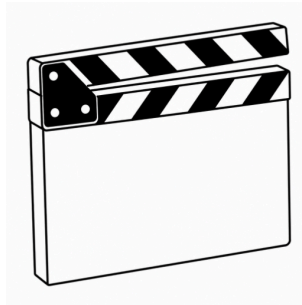
After

Write a 300-word blog post introduction about how small businesses can use AI tools to save time. Write for a non-technical audience. Friendly, conversational tone. No jargon.

Here's the same request with and without the framework. "Write a blog post about AI" — the model will produce something. It might even be decent. But "write a 300-word introduction about how small businesses can use AI to save time, for a non-technical audience, friendly tone, no jargon" — that's a brief. The output is targeted, usable, and a lot closer to done on the first pass. Same task. Completely different results.

Stop asking. Start directing.

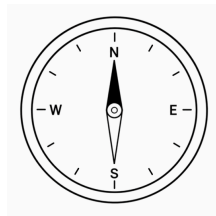
Think brief, not question



The mental shift that makes the biggest difference is this: stop thinking of a prompt as a question and start thinking of it as a brief. When you hire someone to do a job, you don't just say "write something." You tell them the goal, the audience, the format, what to avoid. A prompt is the same thing. You're the director. The model is very capable — but it needs direction, not just a topic.

Key Takeaways

The four elements, every time



Task - be specific about what you want

Context - give it the background it needs

Format - tell it what the output should look like

Constraints - set the guardrails

Four elements. Task, context, format, constraints. You don't always need all four — but when a prompt isn't landing, one of them is almost always missing. Start there. In the next chapter we're going to build on this foundation with specific techniques — few-shot prompting, chain-of-thought, role assignment — and look at when to use each one.